

AI Manager

```
# PERSONAL AI TASK MANAGER
```

```
## Full Product + Architecture Specification for an Implementing LLM
```

```
> Build this as a personal-first, minimalistic AI task manager.
```

```
> The UI should feel like Google Tasks with an intelligent personal planning assistant built into it.
```

```
> The AI is not an add-on chatbot. The AI is the actual planner/manager of the user's day.
```

```
---
```

1. CORE VISION

Build a personal AI-first task management and daily planning application for:

- Desktop/laptop
- Mobile

The product should be extremely minimal and easy to use.

The mental model is:

****Google Tasks + a smart personal chatbot + automatic daily planning + Google Calendar synchronization.****

The application must:

- Display tasks clearly.
- Allow normal task CRUD.
- Synchronize tasks with Google Tasks.
- Synchronize scheduled time blocks with Google Calendar.

- Understand natural-language instructions.
- Create tasks from conversation.
- Edit/delete/complete tasks through conversation.
- Create recurring tasks.
- Break large goals into smaller tasks.
- Estimate task duration.
- Plan the user's day.
- Re-plan when circumstances change.
- Personalize plans using long-term user context.
- Allow the user to explicitly tell the AI what to remember.
- Allow the user to provide a personal system prompt/context.
- Work with the user's own API keys.
- Keep API credentials locally on the user's device.
- Avoid requiring the developer to provide API keys.
- Avoid mandatory authentication.
- Use MongoDB Atlas for cross-device synchronization.
- Remain maintainable and easy to use.
- Support future desktop/mobile updates without requiring a complete uninstall/reinstall.

The application should feel like:

> "My own personal AI that manages my tasks and day."

It should NOT feel like:

> "A complicated productivity platform with an AI chatbot."

2. DESIGN PRINCIPLE

The most important product principle is:

MINIMAL UI, MAXIMUM INTELLIGENCE

The user should see very little complexity.

The AI should handle complexity behind the scenes.

Do not create a Notion-like interface.

Do not create dozens of configuration screens.

Do not expose unnecessary AI terminology to the user.

The user should mostly see:

- What needs to be done
- What is scheduled
- What has been completed
- What the AI recommends
- A simple way to talk to the AI

3. TARGET EXPERIENCE

A typical morning:

User opens the application.

The application says:

> Good morning. What's your plan for today?

User:

> "I have college until 4. I need to finish my DBMS assignment, study DSA for two hours, work on my ML project and go to the gym."

The AI should understand this.

It should:

1. Extract tasks.
2. Identify deadlines if mentioned.
3. Identify duration.
4. Break tasks down if needed.
5. Check existing tasks.
6. Check Google Calendar.
7. Check routines.
8. Retrieve relevant personal memories.
9. Consider user's personal system instructions.
10. Consider goals.
11. Generate a realistic schedule.
12. Detect conflicts.
13. Fix conflicts.
14. Save the tasks.
15. Sync tasks to Google Tasks.
16. Create/update exact time blocks in Google Calendar.
17. Display the resulting day clearly.

4. CORE LOOP

The application should follow this conceptual loop:

REMEMBER

↓

UNDERSTAND

↓

PLAN

↓

SCHEDULE

↓

MONITOR

↓

REPLAN

↓

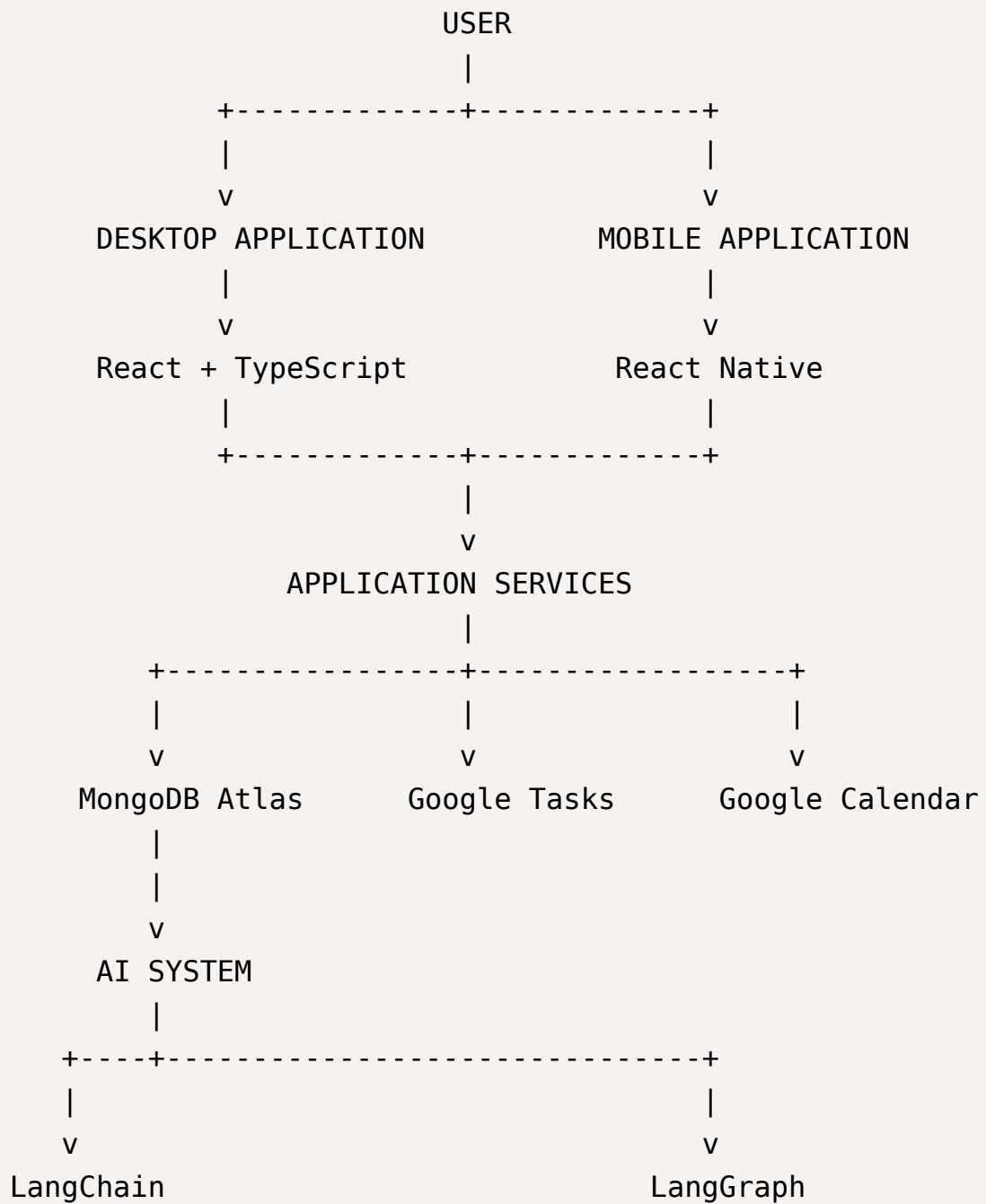
LEARN

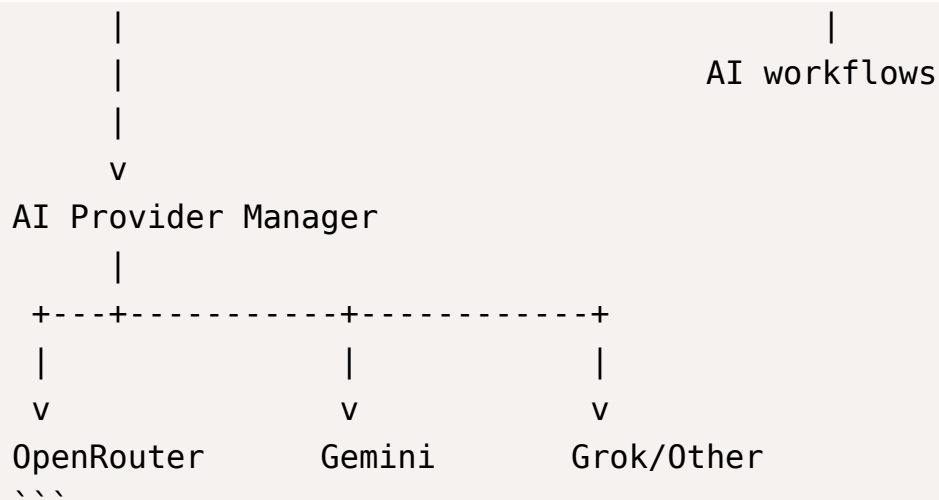


REMEMBER

5. APPLICATION ARCHITECTURE

High-level architecture:





6. IMPORTANT: NO MEM0

Do NOT use Mem0.

The application should implement its own memory system.

MongoDB will store:

- structured application data
- tasks
- goals
- routines
- preferences
- schedules
- personal memories
- memory embeddings

MongoDB Atlas Vector Search will provide semantic retrieval.

7. BYOK / BYOI ARCHITECTURE

The application is designed around:

Bring Your Own Key

The application must NOT contain developer-owned AI API keys.

The user supplies:

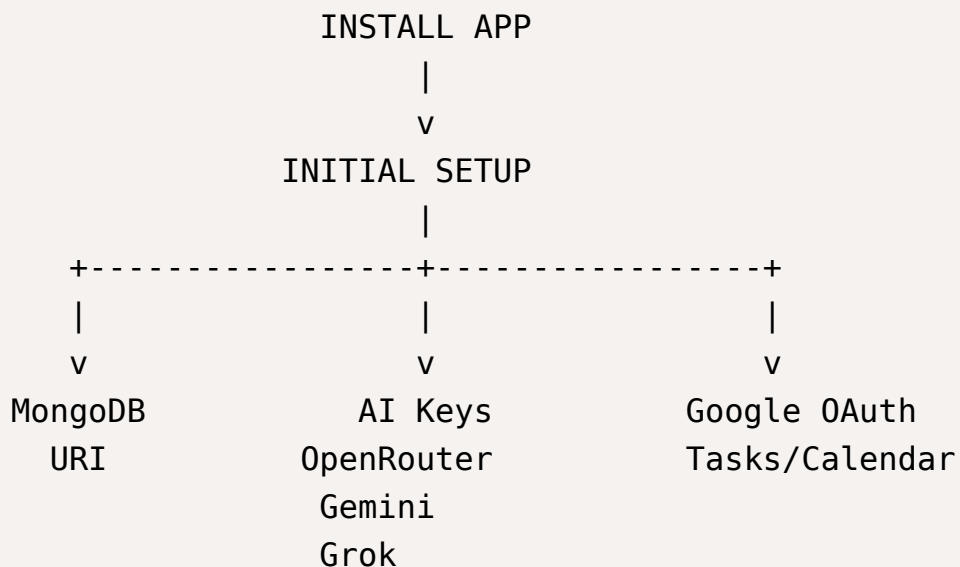
- MongoDB URI
- OpenRouter API key
- Gemini API key
- Grok API key if desired
- other provider keys if added later

For Google Tasks and Google Calendar:

- use Google OAuth
- do not ask for a normal Google API key for user authorization

The application should become the user's own personal instance.

Conceptually:



|
v
PERSONAL INSTANCE

8. CREDENTIAL OWNERSHIP

Credentials belong to the user.

Do NOT:

- hardcode them
- commit them to Git
- put them into public frontend bundles
- store them in MongoDB as plain text
- send them to a developer-controlled database

Store secrets locally using appropriate secure storage.

Desktop:

- OS credential store / secure credential storage

Android:

- Android secure storage / Keystore-backed solution

iOS:

- Keychain

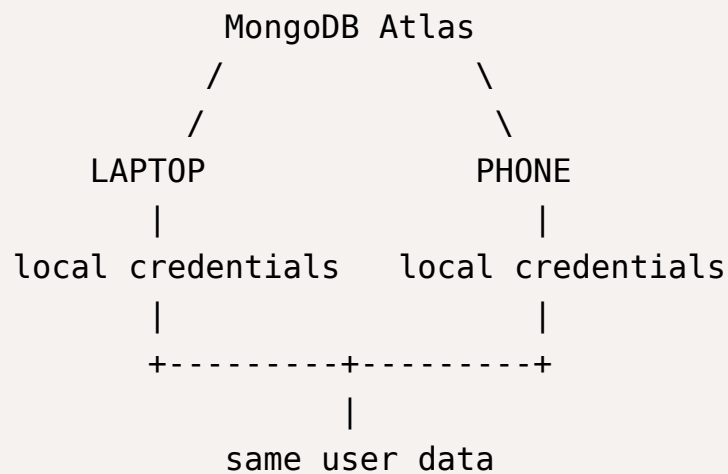
The browser/frontend must not have unrestricted access to secret credentials where avoidable.

9. MONGODB

Use MongoDB Atlas.

MongoDB is responsible for cross-device synchronization and persistent application data.

Both desktop and mobile can use the same database.



Do not store API secrets in MongoDB.

10. AUTHENTICATION

Do NOT add Firebase Authentication by default.

This is a personal/BYOK-oriented application.

There is no requirement for:

- registration
- login
- multi-user SaaS
- account switching
- tenant management

A local installation can be associated with one configured user/profile.

However, design the database cleanly enough that a future m

ulti-user version could be introduced without rewriting the entire system.

11. FIREBASE

Firebase may be explored for useful infrastructure, but do not introduce Firebase Authentication merely because it is popular.

The primary persistent database is MongoDB Atlas.

Avoid unnecessary duplication such as:

Firebase Firestore

+

MongoDB

unless a future requirement clearly justifies it.

12. DESKTOP APPLICATION

The desktop application should NOT require the user to manually start the backend every time.

Bad UX:

Open terminal

↓

npm run backend

↓

npm run frontend

↓

Open browser

↓

Use application

Good UX:

Double-click application

↓

Local services start automatically

↓

Application opens

↓

Ready

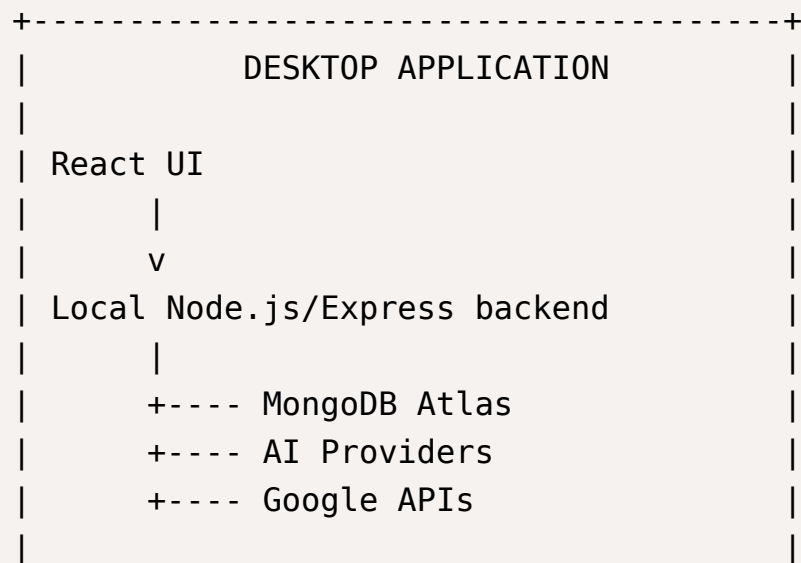
The desktop application can be packaged using a technology such as:

- Tauri
- Electron

Choose based on reliability, bundle size, native integration and ease of maintenance.

A local Node.js/Express backend can run as part of the desktop application.

Architecture:



+-----+

13. MOBILE APPLICATION

The mobile application should be a real mobile application.

Use:

React Native

Potentially Expo if compatible with all required native functionality.

Do NOT depend on an always-running Express server on the phone.

Mobile operating systems restrict continuous background execution.

Instead:

React Native

↓

mobile application services/native modules

↓

MongoDB / AI providers / Google APIs

The user should never need to open a terminal or manually start a backend on mobile.

For normal foreground use:

OPEN APP

↓

USE APP

↓

AI REQUEST

↓

SAVE/SYNC

Background operations must respect Android/iOS restrictions.

14. MOBILE + DESKTOP SHARED LOGIC

Keep shared business logic as much as practical.

Possible monorepo:

apps/

 web/

 desktop/

 mobile/

packages/

 shared-types/

 schemas/

 ai/

 planning/

 domain/

 utilities/

The UI should be platform-specific when necessary, but the following should be shared where practical:

- TypeScript types
- Zod schemas
- task domain models
- AI request/response schemas
- planning rules
- prompts
- provider interfaces

- task state logic

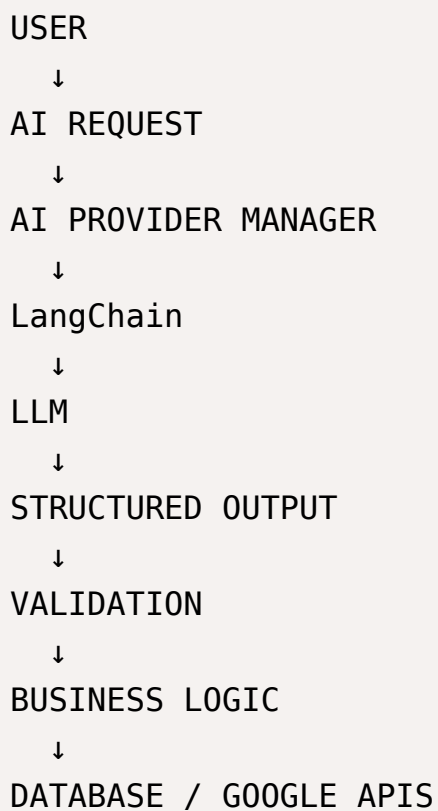
Avoid duplicating the entire business logic separately for desktop and mobile.

15. AI SYSTEM

The AI system consists of:

1. AI Provider Manager
2. LangChain
3. LangGraph
4. Memory system
5. Vector search
6. Planning engine
7. Validation/business rules

Architecture:



16. AI PROVIDER MANAGER

Never hardcode the application to one AI provider.

Create a provider abstraction:

```
interface AIProvider {  
    generate(request): Promise<AIResponse>  
}
```

Implement:

```
OpenRouterProvider  
GeminiProvider  
GrokProvider
```

Future providers should be easy to add.

Application code should call:

```
AIService.generate()
```

rather than directly calling Gemini/OpenRouter throughout the codebase.

17. PROVIDER FALLBACK

Example:

```
AI REQUEST  
  ↓  
OpenRouter  
  ↓
```

```
success → return
    |
    failure/rate limit
    ↓
Gemini
    ↓
success → return
    |
    failure
    ↓
Grok / another enabled provider
```

Implement:

- rate-limit detection
- timeout handling
- limited retries
- provider fallback
- model selection
- error logging
- token usage tracking where available
- optional cost tracking

Do NOT attempt to bypass provider quotas or abuse free accounts.

The objective is graceful degradation.

18. AI MODEL ROUTING

Do not use the strongest model for every operation.

Simple operations may not require an LLM.

Example:

"Add gym at 6 PM."

Can potentially be parsed using deterministic date/time logic.

Moderate:

"I need to finish DBMS and DSA before Friday."

Use a fast/inexpensive model.

Complex:

"I have college until 4, need to study DSA, finish DBMS, work on my ML project, go to the gym and still have some free time. Plan my evening realistically."

Use a stronger model.

Suggested routing:

TASK_PARSE

→ fast/cheap

TASK_CLASSIFICATION

→ fast/cheap

MEMORY_EXTRACTION

→ fast/cheap

PLANNING

→ stronger

REPLANNING

→ stronger

CONFLICT_RESOLUTION

→ stronger

19. LANGCHAIN

LangChain is the AI integration/toolkit layer.

Use it for:

- LLM integrations
- prompt templates
- structured outputs
- embeddings
- retrievers
- tools
- model/provider abstraction where useful

Do not use LangChain simply because it is popular.

Every component should have a concrete purpose.

20. LANGGRAPH

LangGraph is the workflow/orchestration layer for complex AI reasoning.

Do NOT use LangGraph for simple CRUD.

Use normal application code for:

- create task
- edit task
- delete task
- complete task
- fetch tasks
- sync Google Tasks

- fetch calendar
- create calendar event

Use LangGraph for:

- morning planning
- complex task decomposition
- daily schedule generation
- conflict resolution
- automatic replanning
- weekly planning
- goal planning/review

21. EXAMPLE LANGGRAPH DAILY PLANNER

START

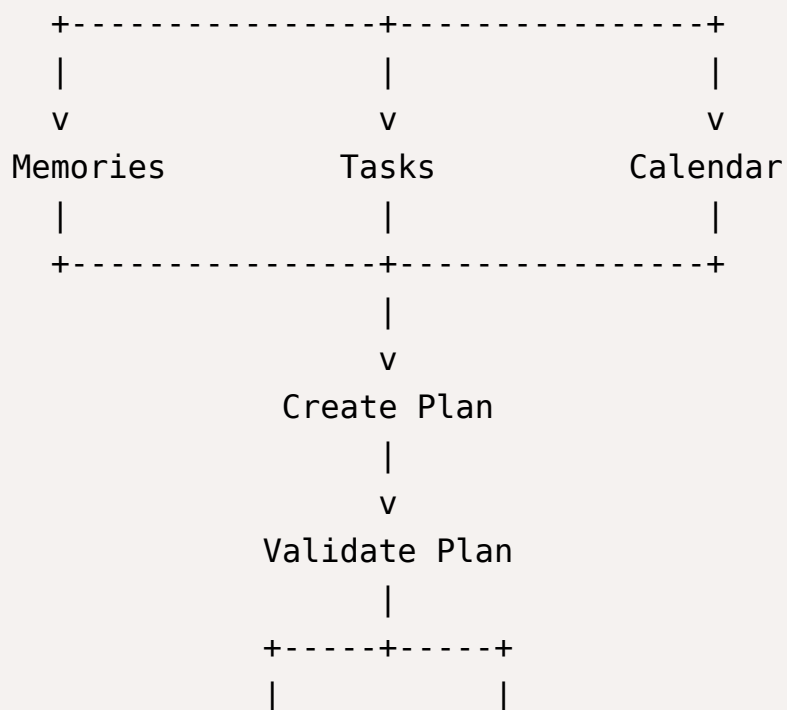
↓

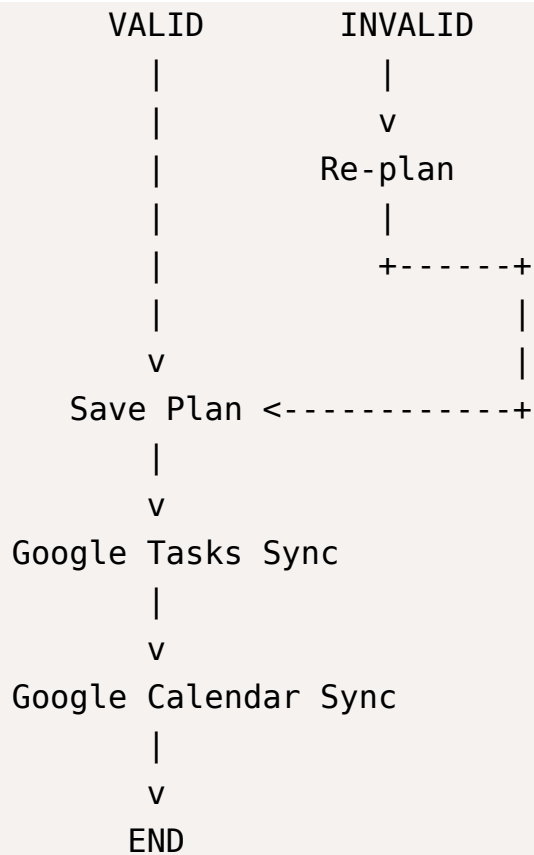
Understand Request

↓

Retrieve Context

↓





22. MEMORY SYSTEM

Build a custom personal memory system.

Useful memory categories:

- profile
- goals
- preferences
- routines
- relationships
- scheduling
- learning
- projects
- constraints
- patterns
- decisions

- temporary_context

Examples:

"I prefer studying difficult subjects in the morning."

"I don't like scheduling demanding work after 10 PM."

"I usually go to the gym in the evening."

"DBMS project is important to me this week."

Only store information that can improve future interactions/planning.

23. USER-CONTROLLED MEMORY

The application must have a clear place where the user can explicitly add something for the AI to remember.

Example UI:

AI Memory

Things you told me to remember

+ Add memory

"Prefer morning study sessions."

"Don't schedule tasks after 10 PM."

"Keep Sunday evening free."

Each memory should be editable/deletable.

The user must be able to inspect what the AI remembers.

This is important because the AI's personalization should be transparent and controllable.

24. AUTOMATIC MEMORY EXTRACTION

The AI may also identify useful information from conversations.

Pipeline:

Conversation

↓

Memory extraction

↓

Is this useful long-term?

↓

YES

↓

Create concise memory

↓

Generate embedding

↓

Store in MongoDB

Do not store:

- greetings
- meaningless small talk
- irrelevant one-off statements
- raw transcripts
- API keys
- passwords
- credentials
- unnecessary sensitive data
- AI assumptions

Memory should be:

- concise
- atomic
- useful
- accurate
- attributable to the user

Temporary statements should not automatically become permanent preferences.

25. USER SYSTEM PROMPT / PERSONAL CONTEXT

Add a dedicated settings section:

"AI Instructions" / "Personal Context"

The user can enter free-form information that should be considered whenever the AI responds or plans.

Example:

"I am a college student. I prefer realistic schedules rather than extremely packed schedules. I need breaks between difficult study sessions. I prefer completing difficult technical work earlier in the day."

This should be treated as high-priority user-provided context.

Store it separately from automatically extracted memories.

Example:

user profile:
 systemPrompt:

"..."

The user should be able to edit it at any time.

26. DIFFERENCE BETWEEN SYSTEM PROMPT AND MEMORY

System/Personal Instructions:

Explicit instructions/context written by the user.

Example:

"Never schedule study after 10 PM."

Memory:

Facts/preferences learned from interactions.

Example:

"User often studies DSA in the morning."

Explicit user instructions should take precedence over inferred memories when there is a conflict.

27. VECTOR SEARCH

Vector search is ONLY needed for semantic retrieval of relevant memories and potentially other future semantic data.

Example:

Stored:

"User prefers difficult technical study in the morning."

Embedding:

text

↓

embedding model

↓

vector

↓

MongoDB

Later:

"Plan my study session tomorrow."

↓

query embedding

↓

MongoDB Atlas Vector Search

↓

retrieve relevant memories

Retrieve only the most relevant memories.

Do NOT send the entire memory database to the LLM.

Target roughly 5–15 relevant memories depending on context.

28. VECTOR SEARCH FLOW

User request

↓

Generate embedding

↓

MongoDB Vector Search

↓
Filter by user
↓
Similarity/ranking
↓
Top relevant memories
↓
AI Context Builder
↓
LLM

Use metadata/category filtering where useful.

29. AI CONTEXT BUILDER

Before planning, construct controlled context from:

- current user request
- user system/personal instructions
- relevant memories
- current tasks
- overdue tasks
- deadlines
- goals
- routines
- Google Calendar events
- current date/time
- timezone
- constraints
- existing schedule

Do NOT dump the whole database into the model.

Conceptually:

USER REQUEST

+
PERSONAL INSTRUCTIONS
+
RELEVANT MEMORIES
+
TASKS
+
GOALS
+
ROUTINES
+
CALENDAR
+
CURRENT TIME
+
CONSTRAINTS
↓
AI CONTEXT
↓
PLANNER

- - -

30. AI SHOULD NOT DIRECTLY CONTROL THE DATABASE

Never allow:

LLM
↓
unrestricted MongoDB access

Instead:

LLM
↓
structured proposed action
↓
Zod validation

↓
business rules
↓
authorized service
↓
MongoDB

The AI proposes.
The application validates and executes.

31. ZOD / STRUCTURED OUTPUT

Use Zod or equivalent runtime schema validation.

Example:

```
{  
  "action": "create_task",  
  "title": "Study DSA",  
  "durationMinutes": 60,  
  "priority": "high"  
}
```

Validate before execution.

Example schema:

```
const TaskActionSchema = z.object({  
  action: z.enum([  
    "create_task",  
    "update_task",  
    "delete_task",  
    "complete_task"  
  ]),  
  title: z.string().optional(),  
  durationMinutes: z.number().positive().optional(),  
})
```

```
    priority: z.enum([
        "low",
        "medium",
        "high"
    ]).optional()
});
```

Do not allow arbitrary AI-generated code or database queries.

32. NATURAL-LANGUAGE TASK MANAGEMENT

The chatbot should be able to:

Create:

"Add study DSA tomorrow."

Update:

"Move DSA to 7 PM."

Delete:

"Remove my gym task."

Complete:

"I finished DBMS."

Recurring:

"Remind me to study DSA every weekday at 7 PM."

Break down:

"Break my ML project into tasks."

Prioritize:

"What's most important today?"

Plan:

"Plan my evening."

Replan:

"I haven't done anything since 5. Replan the rest of my day."

Query:

"What do I still need to do today?"

33. RECURRING TASKS

Recurring tasks are a required feature.

Examples:

"Every weekday at 7 PM, study DSA."

"Every Sunday, plan the week."

"Every month, review my goals."

Store recurrence rules in MongoDB.

Prefer a standard recurrence representation such as RRULE where appropriate.

Example:

```
{
  "recurrence": {
    "enabled": true,
    "rule": "FREQ=WEEKLY;BYDAY=MO,TU,WE,TH,FR"
  }
}
```

The application should generate/manage task instances corre

ctly.

Google Tasks recurrence capabilities should be treated as an integration constraint rather than assumed to exactly match the application's recurrence engine.

The application's own recurrence model remains the source of truth.

34. GOOGLE TASKS ARCHITECTURE

MongoDB:

Source of truth

Google Tasks:

Synchronization layer

Supported synchronization should include:

- create
- update
- delete
- complete
- notes/description
- due date
- subtasks where appropriate
- ordering where appropriate

Important:

Google Tasks API does NOT provide exact time-of-day task scheduling.

Therefore:

MongoDB:

```
scheduledStart
scheduledEnd
```

Google Tasks:
task + due date

Google Calendar:
exact time block

35. GOOGLE CALENDAR ARCHITECTURE

Google Calendar represents exact scheduled time blocks.

Example:

MongoDB task:

```
{
  "title": "Study DSA",
  "scheduledStart": "2026-09-15T18:00:00+05:30",
  "scheduledEnd": "2026-09-15T19:00:00+05:30"
}
```

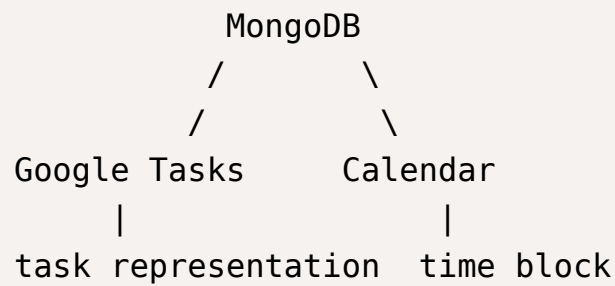
Google Calendar:

18:00–19:00
Study DSA

If the AI moves the task, update the corresponding Calendar event.

If a task is deleted or unscheduled, appropriately update/remove the corresponding Calendar event.

36. GOOGLE SYNC MODEL



MongoDB remains the source of truth.

External APIs are synchronized representations.

If external synchronization fails:

- Do not lose the MongoDB task.
- Mark synchronization status.
- Retry later.
- Inform the user only when useful.

37. SYNC METADATA

Each task can contain:

google:

- taskId
- taskListId
- lastSyncedAt

calendar:

- eventId
- calendarId
- lastSyncedAt

sync status should allow:

synced
pending
failed

Avoid infinite synchronization loops.

Use updatedAt/version/source metadata to detect changes.

38. DAILY PLANNING

The planner should consider:

- fixed calendar events
- task deadlines
- task priority
- estimated duration
- goals
- routines
- user preferences
- personal instructions
- current energy/preferences if available
- breaks
- available time
- realistic workload

Do not simply fill every minute of the day.

The planner should leave reasonable buffers.

The user should be able to configure planning preferences.

39. PERSONALIZATION

Personalization is a core feature.

The same task list should produce different schedules for different users because preferences/context differ.

For example:

User A:

- prefers mornings
- needs long study sessions
- likes large uninterrupted blocks

User B:

- prefers evenings
- likes 30-minute sessions
- needs frequent breaks

The planner should reflect the user's personal context.

The user should feel:

"This plan was made for me."

40. AUTOMATIC REPLANNING

Example:

Original:

16:00–17:00 DSA
17:00–18:00 Gym
18:30–20:00 DBMS
20:00–21:00 ML

User says at 17:30:

"I haven't started DSA."

System:

Current time

+

remaining tasks

+

calendar

+

deadlines

+

memories

+

preferences

↓

Replanner

↓

validate

↓

new schedule

↓

MongoDB

↓

Google Calendar

↓

UI

The planner should adapt instead of forcing the user to manually rearrange everything.

41. CHATBOT

The chatbot is a primary application feature.

It should not be a generic ChatGPT clone.

It must have access to controlled application tools.

Potential tools:

```
get_tasks
create_task
update_task
delete_task
complete_task
create_recurring_task
get_calendar
create_calendar_event
update_calendar_event
delete_calendar_event
get_goals
get_routines
search_memories
save_memory
update_memory
generate_daily_plan
replan_day
```

The AI should call tools through controlled interfaces.

It should never receive unrestricted database access.

42. CHATBOT TABS / APPLICATION NAVIGATION

Keep tabs minimal.

Recommended primary navigation:

1. TODAY
2. TASKS
3. CHAT
4. CALENDAR

Optional secondary/settings area:

- 5. GOALS
- 6. MEMORY
- 7. SETTINGS

Do not put all of these in a cluttered permanent navigation bar on mobile.

For mobile, consider:

Today | Tasks | Chat | Calendar

and place Goals, Memory and Settings inside a profile/settings area.

43. TODAY TAB

This is the primary screen.

Display:

- today's date
- short greeting
- current schedule
- tasks
- completed tasks
- overdue tasks where relevant
- AI-generated timeline
- quick add
- "Ask AI" / chat entry
- Replan button

Example:

TODAY

Good morning

What's your plan for today?

09:00 – College

16:30 – DBMS assignment

18:00 – DSA

19:30 – Gym

21:00 – ML project

[+ Add task]

[Replan my day]

Keep this screen extremely clean.

44. TASKS TAB

Simple task list.

Features:

- inbox
- today
- upcoming
- completed
- recurring tasks
- priorities
- subtasks

Avoid unnecessary project-management features.

Task actions should be fast.

45. CHAT TAB

Dedicated AI conversation.

The user can ask:

"What should I do now?"

"Move my DSA session."

"Plan tomorrow."

"Why did you schedule DBMS before DSA?"

"Remember that I prefer shorter sessions."

"Forget that preference."

"Create a recurring gym task every Monday, Wednesday and Friday."

The AI should execute appropriate actions rather than only respond with text.

46. CALENDAR TAB

Minimal timeline/calendar view.

Show:

- Google Calendar events
- AI scheduled tasks
- free time

- conflicts

Distinguish fixed commitments from flexible tasks.

Avoid rebuilding Google Calendar.

The application should primarily provide a lightweight task-focused schedule view.

47. GOALS TAB

Optional but useful.

Show:

- active goals
- progress
- milestones
- related tasks

Example:

Learn Machine Learning
35%

Learn Linear Algebra ✓
Learn Regression
Build ML project

Goals feed the planner.

48. MEMORY TAB

Show what the AI remembers.

Sections:

- Preferences
- Routines
- Goals
- Personal context
- Explicit memories
- Automatically learned memories

Allow:

- add
- edit
- delete
- disable

This gives the user control over personalization.

49. SETTINGS TAB

Include:

AI Providers

- OpenRouter
- Gemini
- Grok
- other providers

Google

- Tasks connection
- Calendar connection

Database

- MongoDB connection status

AI Instructions

- Personal system prompt

Planning

- preferred working hours
- break preferences
- default buffers
- scheduling preferences

Memory

- memory behavior controls

Sync

- synchronization status

Application

- version
- update status
- diagnostics/logs if necessary

Keep settings organized and simple.

50. MINIMAL UI DESIGN

Visual style:

- clean
- spacious
- calm
- minimal
- fast
- mobile-first
- keyboard-friendly on desktop

Avoid:

- dashboards full of graphs
- excessive colors
- unnecessary cards
- gamification
- complicated project hierarchies
- dozens of filters
- unnecessary animations
- intrusive AI elements

The AI should feel integrated into the product.

51. RESPONSIVENESS

The application should feel instant for ordinary operations.

When the user:

- checks a task
- opens today's list
- edits a title
- navigates tabs

do not wait for an LLM.

AI operations can show a small progress indicator.

Use optimistic UI where safe.

Example:

User checks task

↓

UI immediately marks complete

↓

MongoDB update

↓
Google sync
↓
If sync fails, show subtle status

52. MONGODB COLLECTIONS

Recommended collections:

users
tasks
goals
routines
memories
schedules
calendarEvents
syncMetadata

Potential future collections:

chatSessions
chatMessages
planningRuns
providerUsage
notifications

Do not create collections unnecessarily.

53. MONGODB JSON STRUCTURE

These are example document shapes.

They are schemas, not literal data that must be copied unchanged.

USER

```
{
  "_id": "ObjectId",
  "profile": {
    "name": "User Name",
    "timezone": "Asia/Kolkata"
  },

  "aiInstructions": {
    "systemPrompt": "User-provided personal context and instructions..."
  },

  "planningPreferences": {
    "preferredStartTime": "07:00",
    "preferredEndTime": "22:00",
    "defaultBufferMinutes": 10,
    "preferRealisticSchedules": true,
    "maxContinuousWorkMinutes": 90
  },

  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

TASK

```
{
  "_id": "ObjectId",

  "userId": "ObjectId",

  "title": "Study DSA",
```

```
"description": "Complete graph algorithms revision",

"status": "pending",

"priority": "high",

"parentTaskId": null,

"goalId": null,

"estimatedMinutes": 60,

"actualMinutes": null,

"dueAt": "ISODate",

"scheduledStart": "ISODate",

"scheduledEnd": "ISODate",

"tags": [
  "DSA",
  "study"
],

"source": "user",

"recurrence": {
  "enabled": false,
  "rule": null,
  "timezone": "Asia/Kolkata"
},

"google": {
  "taskId": "google-task-id",
  "taskListId": "google-task-list-id",
  "lastSyncedAt": "ISODate",
  "syncStatus": "synced"
```

```

    },

    "calendar": {
        "eventId": "google-calendar-event-id",
        "calendarId": "primary",
        "lastSyncedAt": "ISODate",
        "syncStatus": "synced"
    },

    "createdAt": "ISODate",
    "updatedAt": "ISODate",
    "completedAt": null
}

---

## GOAL

{
    "_id": "ObjectId",

    "userId": "ObjectId",

    "title": "Learn Machine Learning",

    "description": "Build strong ML fundamentals and practical projects.",

    "status": "active",

    "priority": "high",

    "targetDate": "ISODate",

    "progress": 0,

    "milestones": [
        {

```

```

        "_id": "ObjectId",
        "title": "Learn linear algebra",
        "status": "completed"
    },
    {
        "_id": "ObjectId",
        "title": "Learn regression",
        "status": "pending"
    }
],

"createdAt": "ISODate",
"updatedAt": "ISODate"
}

---

## ROUTINE

{
    "_id": "ObjectId",

    "userId": "ObjectId",

    "title": "College",

    "type": "fixed",

    "daysOfWeek": [
        1,
        2,
        3,
        4,
        5
    ],

    "startTime": "09:00",

```

```
"endTime": "16:00",

"timezone": "Asia/Kolkata",

"flexible": false,

"createdAt": "ISODate",
"updatedAt": "ISODate"
}

---

## MEMORY

{
  "_id": "ObjectId",

  "userId": "ObjectId",

  "category": "preferences",

  "content": "Prefers studying difficult technical subjects
in the morning.",

  "embedding": [
    0.021,
    -0.184,
    0.731
  ],

  "importance": 0.8,

  "confidence": 0.95,

  "source": "explicit_user",

  "status": "active",
```

```
"createdAt": "ISODate",  
  
"updatedAt": "ISODate",  
  
"lastUsedAt": "ISODate"  
}
```

Possible source values:

```
explicit_user  
conversation  
manual  
system_import
```

SCHEDULE

```
{  
  "_id": "ObjectId",  
  
  "userId": "ObjectId",  
  
  "date": "2026-09-15",  
  
  "items": [  
    {  
      "_id": "ObjectId",  
      "taskId": "ObjectId",  
      "start": "ISODate",  
      "end": "ISODate",  
      "type": "task"  
    },  
    {  
      "_id": "ObjectId",  
      "title": "College",  
      "start": "ISODate",  
      "end": "ISODate",  
    }  
  ]  
}
```

```

        "type": "calendar"
    }
],

"generatedBy": "ai",

"version": 3,

"createdAt": "ISODate",
"updatedAt": "ISODate"
}

---

## SYNC METADATA

{
    "_id": "ObjectId",

    "userId": "ObjectId",

    "googleTasks": {
        "connected": true,
        "lastSyncAt": "ISODate",
        "status": "healthy"
    },

    "googleCalendar": {
        "connected": true,
        "lastSyncAt": "ISODate",
        "status": "healthy"
    }
}

```

Do not store raw OAuth secrets here unless they are encrypted and there is a compelling reason.

Prefer device secure storage.

54. CHAT DATA

If chat history is persisted, use a structure such as:

```
{
  "_id": "ObjectId",
  "userId": "ObjectId",
  "title": "Daily planning",
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

Messages:

```
{
  "_id": "ObjectId",
  "sessionId": "ObjectId",
  "role": "user",
  "content": "Plan my evening.",
  "createdAt": "ISODate"
}
```

Avoid treating the entire chat history as memory.

Chat history and long-term memory are different concepts.

55. PROVIDER CONFIGURATION

Do NOT store raw API keys in MongoDB.

Local non-secret configuration:

```
{
```

```

"providers": {
  "openrouter": {
    "enabled": true,
    "defaultModel": "..."
  },

  "gemini": {
    "enabled": true,
    "defaultModel": "..."
  },

  "grok": {
    "enabled": false,
    "defaultModel": "..."
  }
}

```

Actual API keys:

DEVICE SECURE STORAGE

56. DATABASE INDEXING

Design indexes around real queries.

At minimum consider:

tasks:

- userId + status
- userId + dueAt
- userId + scheduledStart
- userId + updatedAt

memories:

- userId + category

vector index for embedding retrieval

goals:

userId + status

routines:

userId

schedules:

userId + date

Avoid adding indexes blindly.

57. SCALABILITY

Even though the initial application is personal, architecture should be clean enough to scale.

Potential future:

Version 1:

single user

Version 2:

multiple users

Version 3:

cloud backend / optional hosted service

Version 4:

team/family features

Do not build multi-user infrastructure now if it adds unnecessary complexity.

But keep:

userId
timestamps
versioning
clean service boundaries

so migration is possible later.

58. APPLICATION UPDATE STRATEGY

The application must support future updates without requiring a full uninstall/reinstall.

Desktop:

Use an application packaging/update mechanism that supports:

- version detection
- incremental update
- automatic or user-approved updates
- migration scripts for local data/configuration
- rollback/failure handling where practical

Mobile:

Use standard app-store update mechanisms or an appropriate supported update mechanism.

The user should retain:

- MongoDB configuration
- AI provider configuration
- Google connection
- personal memories
- tasks

- preferences

when updating the application.

Do NOT tie user data to an app version.

59. DATABASE MIGRATIONS

Whenever the MongoDB schema changes:

Old version

↓

migration

↓

new version

Store a schema/application version.

Example:

```
{
  "schemaVersion": 3
}
```

The application should migrate old documents safely.

Do not require users to delete their data when the app updates.

60. LOCAL CONFIGURATION MIGRATION

When the application updates:

Version 1 config

↓
detect old format
↓
migrate
↓
Version 2 config

Credentials should remain in secure storage.

Never require users to re-enter every API key simply because the UI was updated.

61. OFFLINE-FIRST DIRECTION

The application should eventually maintain a local cache for responsiveness.

Concept:

UI
↓
local cache
↓
MongoDB synchronization

However, do not over-engineer offline synchronization in the first version.

First achieve reliable:

- online operation
- synchronization
- conflict handling

Then add sophisticated offline behavior.

62. FAILURE HANDLING

The app must remain usable if:

- OpenRouter fails
- Gemini fails
- Grok fails
- MongoDB temporarily fails
- Google Tasks fails
- Google Calendar fails
- AI returns malformed output

Examples:

OpenRouter

↓

failure

↓

Gemini

↓

success

Calendar failure:

Task saved in MongoDB

↓

calendar sync pending/failed

↓

retry later

Never lose user data because an external API failed.

63. SECURITY PRINCIPLES

Never:

- hardcode API keys
- commit API keys
- store plaintext secrets in MongoDB
- expose unrestricted database credentials to frontend code
- give the LLM arbitrary database access
- allow arbitrary code execution from AI output

Always:

- validate AI output
- use secure credential storage
- use least-privilege database credentials
- use OAuth for Google
- sanitize/validate user inputs
- log errors without logging secrets

64. API / BACKEND LAYER

Desktop backend can use:

Node.js

+

TypeScript

+

Express

Possible routes:

/api/tasks

/api/goals

/api/routines

/api/memory

/api/planning

/api/chat

```
/api/google/tasks  
/api/google/calendar  
/api/settings
```

But keep the route layer thin.

Use services:

```
TaskService  
GoalService  
RoutineService  
MemoryService  
PlanningService  
GoogleTasksService  
GoogleCalendarService  
AIService  
SyncService
```

65. AI SERVICE LAYER

Suggested:

```
AIService  
  |  
  +-- ProviderManager  
  |  
  +-- PromptManager  
  |  
  +-- EmbeddingService  
  |  
  +-- StructuredOutputValidator  
  |  
  +-- UsageTracker  
  
ProviderManager  
  |
```

```
+-- OpenRouterProvider
+-- GeminiProvider
+-- GrokProvider
```

```
---
```

66. PLANNING SERVICE

PlanningService should orchestrate:

- current tasks
- goals
- routines
- calendar
- memories
- user instructions
- current time
- constraints

For complex reasoning it invokes LangGraph.

Example:

```
PlanningService
  ↓
DailyPlannerGraph
  ↓
retrieve context
  ↓
generate plan
  ↓
validate
  ↓
replan if necessary
  ↓
return final plan
  ↓
save
```

67. BUSINESS RULES

Examples:

A task cannot overlap a fixed calendar event.

A task cannot have an end before its start.

Duration must be positive.

Completed tasks should not be scheduled again unless explicitly requested.

A deadline must not be silently ignored.

User explicit instructions have priority over inferred preferences.

AI-generated schedules must be validated before saving.

Google synchronization must not overwrite newer local changes blindly.

68. TOOL-BASED AI

Give the chatbot controlled tools.

Example:

```
getTodayTasks()  
getUpcomingTasks()  
createTask()  
updateTask()
```

```
deleteTask()
completeTask()
createRecurringTask()
getCalendarEvents()
createCalendarEvent()
updateCalendarEvent()
deleteCalendarEvent()
searchMemory()
saveMemory()
updateMemory()
deleteMemory()
getGoals()
getRoutines()
planDay()
replanDay()
```

Each tool should have:

- input schema
- authorization check
- validation
- clear return type
- error handling

69. EXAMPLE END-TO-END REQUEST

User:

"Add a recurring task to study DSA every weekday at 7 PM for one hour."

Pipeline:

React

↓

Chat API

```
↓
LangGraph/appropriate workflow
↓
Understand intent
↓
Structured action
↓
Zod validation
↓
Create recurrence
↓
Save task/recurrence in MongoDB
↓
Google Tasks synchronization where supported
↓
Create/update appropriate Calendar schedule
↓
Response:

"Done. I've scheduled DSA study for weekdays at 7 PM."

---
```

70. EXAMPLE PERSONALIZED PLANNING

User says:

"Plan tomorrow."

Context:

Personal instruction:

"Don't schedule demanding work after 10 PM."

Memory:

"Prefers difficult technical work in the morning."

Routine:

"College 9–4."

Calendar:

"Gym 7–8 PM."

Tasks:

"DBMS deadline tomorrow."

Goal:

"Learn ML."

Planner:

09:00–16:00 College

16:30–17:30 DBMS

17:30–18:00 Break

18:00–19:00 ML

19:00–20:00 Gym

20:30–21:30 DSA

21:30 onward Free

The actual schedule should depend on real data.

The point is that the AI should use personal context rather than generating a generic schedule.

71. WHY THE APPLICATION IS NOT JUST A CHATBOT

The chatbot is only the interface to the intelligence.

The actual system contains:

Chat

+

Task database

+

Memory
+
Calendar
+
Goals
+
Planning engine
+
Synchronization
+
Provider manager

The user should be able to accomplish everything through either:

1. Direct UI
2. Natural-language AI

Example:

Direct UI:
[✓] Study DSA

AI:
"I finished DSA."

Both should result in the same underlying task state.

72. RECOMMENDED TECH STACK

Frontend:
React
TypeScript
Tailwind CSS

Desktop:

Tauri or Electron

Node.js

Express

Mobile:

React Native

Expo where appropriate

Database:

MongoDB Atlas

MongoDB Atlas Vector Search

AI:

LangChain

LangGraph

Providers:

OpenRouter

Gemini

Grok if available/appropriate

future providers through abstraction

Validation:

Zod

Google:

Google Tasks API

Google Calendar API

Google OAuth 2.0

Credential storage:

OS secure storage / Android Keystore / iOS Keychain

Development:

Git

GitHub

TypeScript

ESLint

Prettier

73. RECOMMENDED PROJECT STRUCTURE

```
project/
├── apps/
│   ├── desktop/
│   │   ├── src/
│   │   └── native/
│   └── mobile/
│       ├── screens/
│       ├── components/
│       └── native/
├── packages/
│   ├── shared-types/
│   ├── schemas/
│   ├── domain/
│   ├── ai/
│   │   ├── providers/
│   │   ├── prompts/
│   │   ├── embeddings/
│   │   └── graphs/
│   ├── planning/
│   └── utilities/
├── backend/
│   ├── src/
│   │   ├── server.ts
│   │   ├── config/
│   │   ├── db/
│   │   │   ├── connection.ts
│   │   │   └── models/
```

```

|
|
|   ├── routes/
|   ├── services/
|   |   ├── task.service.ts
|   |   ├── goal.service.ts
|   |   ├── routine.service.ts
|   |   ├── memory.service.ts
|   |   ├── planning.service.ts
|   |   ├── sync.service.ts
|   |   ├── googleTasks.service.ts
|   |   └── googleCalendar.service.ts
|   └── ai/
|       ├── ai.service.ts
|       ├── provider-manager.ts
|       ├── providers/
|       |   ├── openrouter.ts
|       |   ├── gemini.ts
|       |   └── grok.ts
|       ├── prompts/
|       ├── embeddings/
|       └── graphs/
|           ├── daily-planner.ts
|           ├── replanner.ts
|           └── conflict-resolver.ts
|
|   └── package.json
|
|── tests/
|
|── README.md
|
└── package.json

```

Simplify this structure if it becomes unnecessarily complex.

74. DEVELOPMENT PHASES

Do not implement everything at once.

PHASE 1

Minimal React UI

PHASE 2

Node + Express local backend

PHASE 3

MongoDB Atlas

PHASE 4

Task CRUD

PHASE 5

Google Tasks

PHASE 6

Google Calendar

PHASE 7

One AI provider

PHASE 8

AI natural-language task creation/editing

PHASE 9

Recurring tasks

PHASE 10

Task decomposition

PHASE 11

Daily planning

PHASE 12

Zod validation + deterministic planner validation

PHASE 13

LangGraph planner

PHASE 14

Custom memory

PHASE 15

Embeddings + MongoDB Vector Search

PHASE 16

Personal system prompt

PHASE 17

Automatic replanning

PHASE 18

Multiple AI providers/fallback

PHASE 19

Desktop packaging + automatic backend startup

PHASE 20

Mobile application

PHASE 21

Secure mobile credential storage

PHASE 22

Cross-device polish/offline cache/sync conflict handling

PHASE 23

Application update/migration system

Do not prematurely build complex agent infrastructure.

75. TESTING REQUIREMENTS

Test independently:

Task CRUD

Google Tasks synchronization

Google Calendar synchronization

Recurrence

Memory creation

Memory retrieval

Vector search

AI structured output

AI provider fallback

Planning

Replanning

Conflict detection

Credential storage

Database migration

Application update

Cross-device synchronization

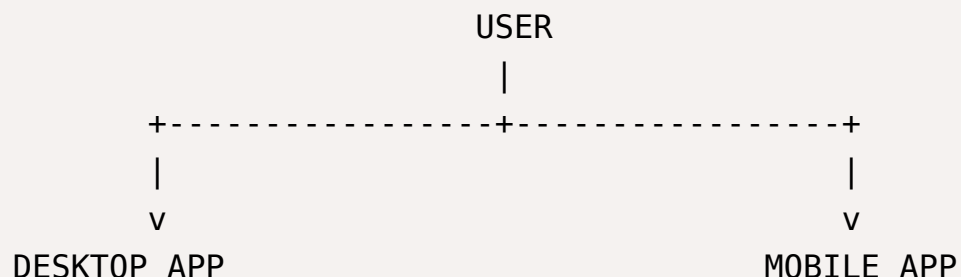
AI behavior should have deterministic tests around its outputs/actions.

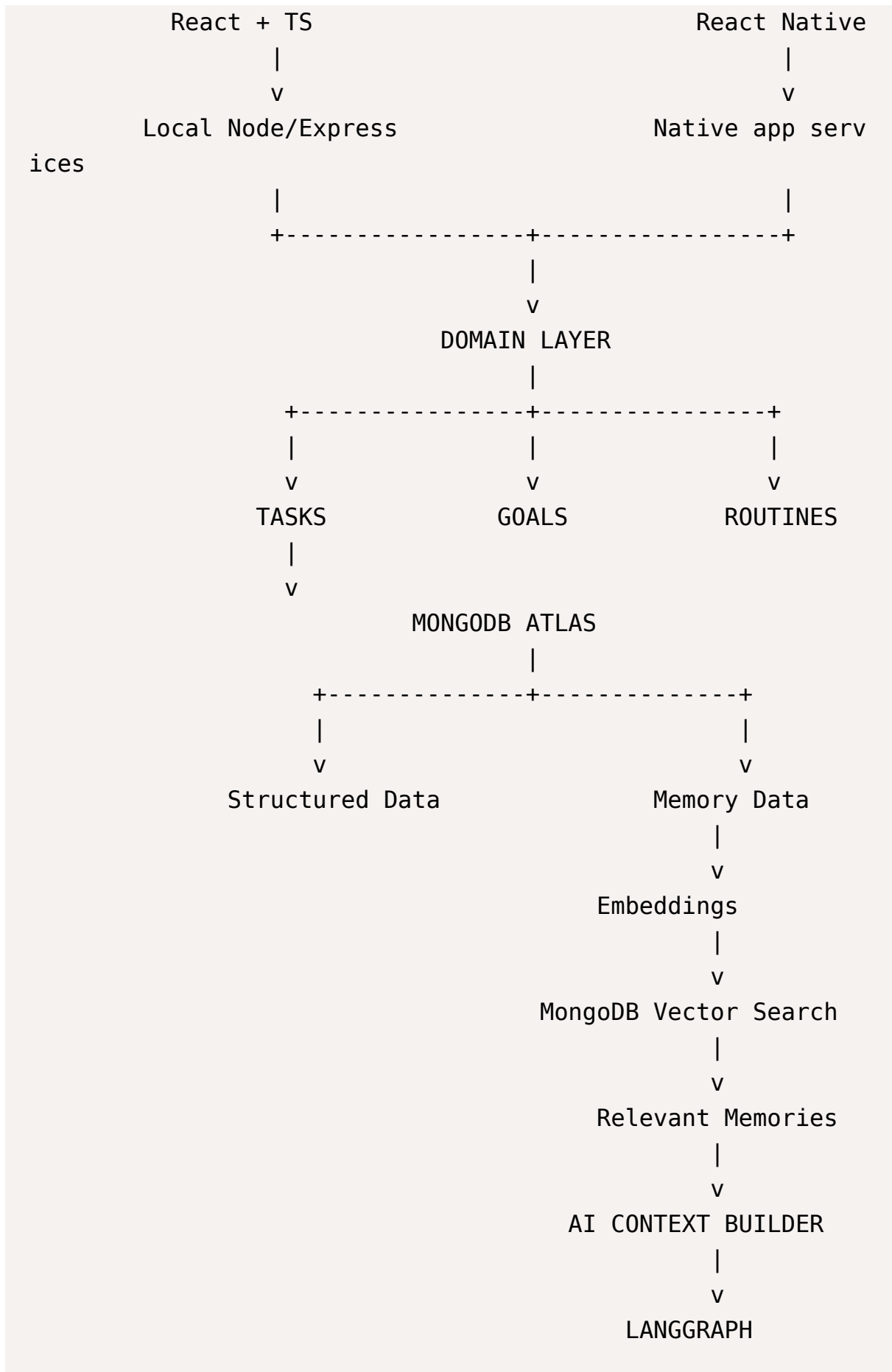
76. IMPORTANT ARCHITECTURAL RULES

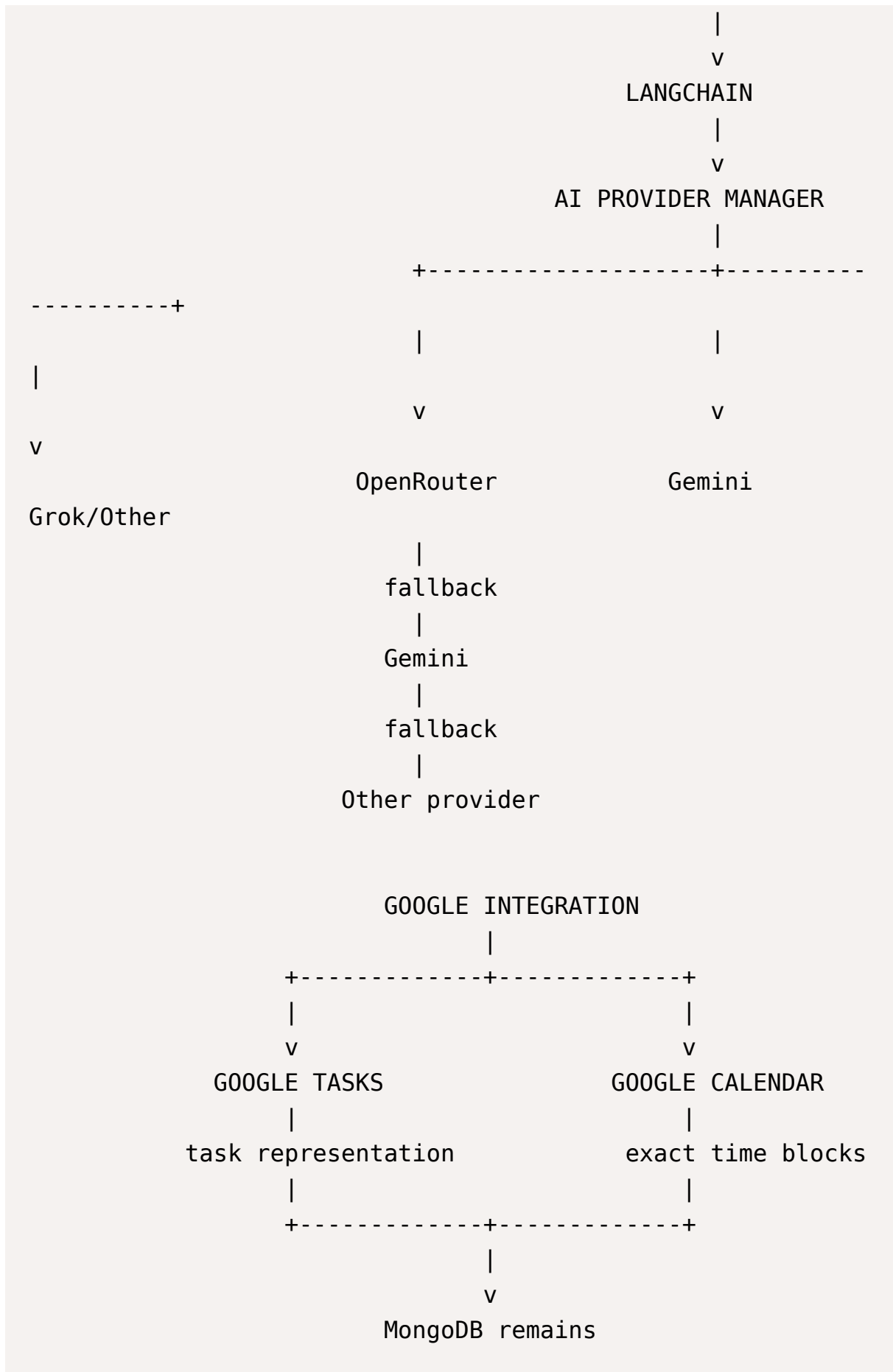
1. No Mem0.
2. No mandatory authentication.
3. No developer-owned AI API keys.
4. No plaintext API keys in MongoDB.
5. No unrestricted MongoDB access from the frontend.
6. No unrestricted database access for the LLM.
7. MongoDB is the application source of truth.
8. Google Tasks is a synchronization layer.

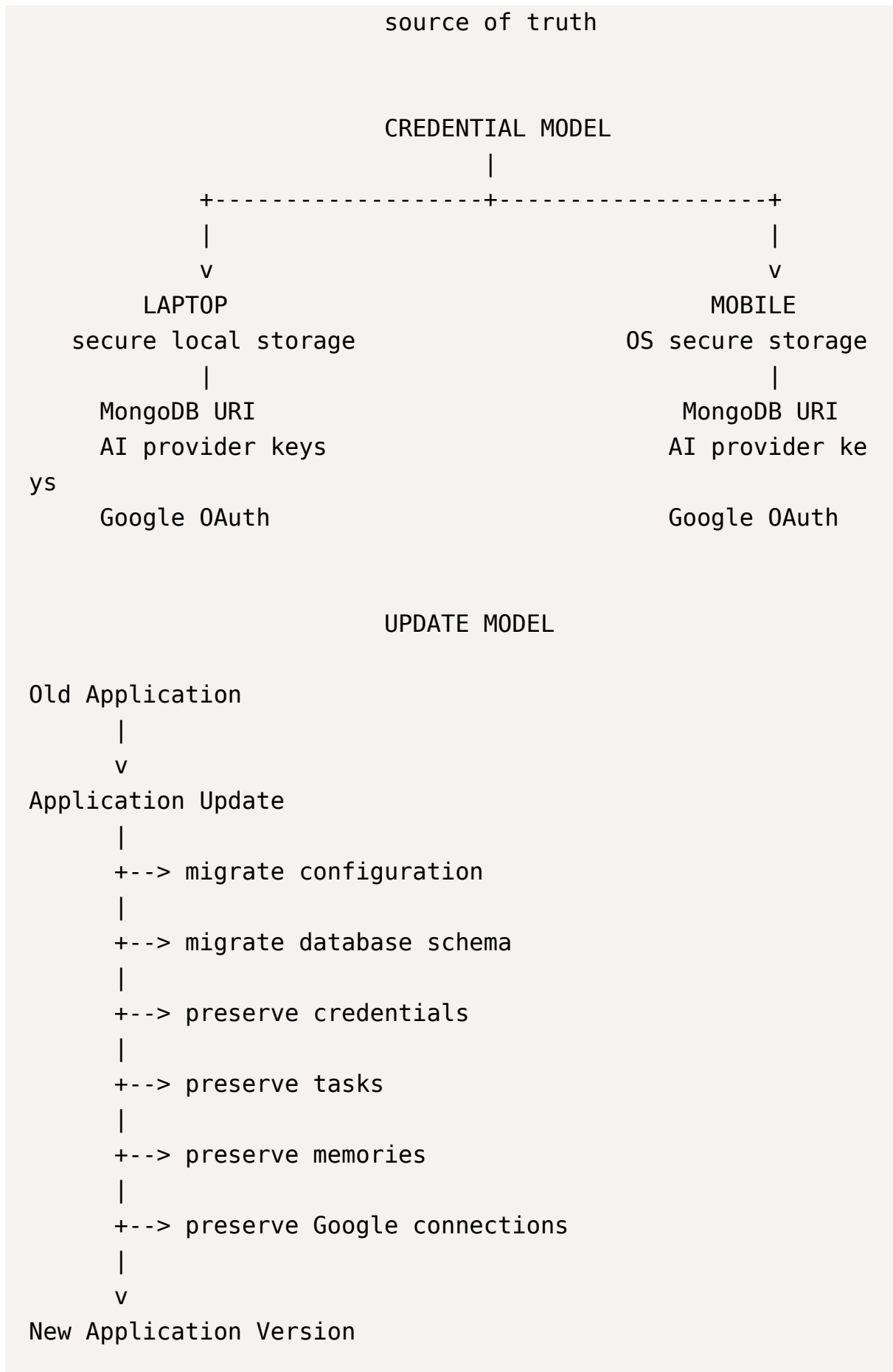
9. Google Calendar represents exact time blocks.
10. Google Tasks cannot represent exact task time-of-day through its API.
11. LangChain is the AI integration/toolkit layer.
12. LangGraph is the complex workflow layer.
13. Vector search is primarily for semantic memory retrieval.
14. Personal instructions are separate from learned memories.
15. Explicit user instructions take precedence over inferred memories.
16. Deterministic logic stays outside the LLM.
17. AI proposes; backend validates and executes.
18. Provider failures must degrade gracefully.
19. Do not depend on unlimited free AI tokens.
20. Keep UI extremely minimal.
21. Do not create unnecessary productivity features.
22. Desktop backend must start automatically with the packaged app.
23. Mobile must not require manually starting a backend.
24. User configuration/data must survive application updates.
25. Schema migrations must preserve user data.
26. Desktop and mobile should reuse shared domain/AI logic where practical.
27. External API failures must not cause local task loss.
28. Build incrementally.

77. FINAL PRODUCT ARCHITECTURE









The user should NOT need to uninstall and reinstall the application just because a new version is released.

78. FINAL INSTRUCTION TO THE IMPLEMENTING LLM

You are designing and implementing the application described in this document.

Treat this document as the architectural source of truth.

Before implementing a feature:

1. Determine whether it genuinely supports the product vision.
2. Prefer the simplest architecture that solves the problem.
3. Do not introduce unnecessary services.
4. Do not introduce Mem0.
5. Do not introduce authentication unless explicitly requested.
6. Do not hardcode API keys.
7. Do not store API keys in MongoDB.
8. Protect credentials using secure local storage.
9. Keep MongoDB as the application source of truth.
10. Keep Google Tasks and Google Calendar synchronized with the application.
11. Remember that Google Tasks does not provide exact task time-of-day through its API.
12. Use Google Calendar for exact scheduled time.
13. Use LangChain only where it provides concrete AI integration value.
14. Use LangGraph for genuinely multi-step reasoning workflows.
15. Use MongoDB Vector Search for semantic memory retrieval.

16. Keep explicit user instructions separate from inferred memory.
17. Validate every AI-generated action before execution.
18. Never allow the LLM unrestricted database access.
19. Keep deterministic business logic deterministic.
20. Implement AI provider fallback without attempting to by pass quotas.
21. Make the application fast and responsive.
22. Keep the interface minimal.
23. Make the chatbot an integrated task-management interface, not a generic chat page.
24. Support recurring tasks.
25. Support personalized planning.
26. Support explicit user memories.
27. Support automatic useful-memory extraction.
28. Support daily planning and automatic replanning.
29. Support desktop and mobile.
30. Make desktop backend startup automatic.
31. Do not require a manually started backend on mobile.
32. Preserve all user configuration and data across application updates.
33. Build migrations for schema/configuration changes.
34. Design clean interfaces so future providers/platforms can be added.
35. Do not over-engineer the first version.

Most importantly:

****The application must help the user, not become another thing the user has to manage.****

The final experience should be:

OPEN APP

↓

SEE TODAY

↓

TELL AI WHAT NEEDS TO HAPPEN

↓
AI UNDERSTANDS PERSONAL CONTEXT
↓
AI PLANS
↓
TASKS + GOOGLE TASKS + GOOGLE CALENDAR UPDATED
↓
USER LIVES THEIR DAY
↓
IF PLANS CHANGE → AI REPLANS

The interface stays simple.

The intelligence stays deep.